

TECHNICAL RESEARCH REPORT

技术研究报告

AI Python 电路绘图库 深度横评报告

难点剖析 · 最新进展 · MCU 板卡级原理图稳定产出方案

本报告系统梳理 SKiDL、atopile、kicad-sch-api 等 Python 电路绘图库的设计哲学与工程能力，结合 Schemato、ChipNeMo、Netlistify、SchGen 等最新学术成果，深入剖析 AI 绘制电路原理图的核心难点，并给出基于嘉立创 EDA (lceda) 等工具稳定产出 MCU 板卡级原理图的工程化路径。

SKiDL

atopile

LLM4EDA

lceda

MCU

RESEARCH SCOPE

Python 库 · 学术论文 · 工业工具

METHODOLOGY

文献综述 + 工程实测 + 方案设计

TARGET AUDIENCE

嵌入式工程师 / EDA 研究者

DELIVERABLE

可复用的 MCU 原理图生成流水线

目录

- 第一章 引言：AI 电路绘图的兴起与挑战 3**
 - 1.1 从手绘到代码化：电路设计范式的三次跃迁 3
 - 1.2 AI 介入电路设计的契机与瓶颈 3
 - 1.3 本报告的研究范围与方法 4
- 第二章 Python 电路绘图库横评 5**
 - 2.1 SKiDL: Pythonic 的网表生成器 5
 - 2.2 atopile: 声明式硬件描述语言 5
 - 2.3 kicad-sch-api: 直接操作 KiCad 原理图 6
 - 2.4 其他相关工具 7
 - 2.4.1 SchemDraw: 教学级原理图绘制 7
 - 2.4.2 PySpice: SPICE 仿真接口 7
 - 2.4.3 KiCad 官方 Python API (kicad-cli + IPC API) 7
 - 2.5 横评对比表 7
- 第三章 AI 绘制电路原理图的核心难点 9**
 - 3.1 原理图的双重语义：拓扑与几何 9
 - 3.2 元件库与符号库的标准化缺失 9
 - 3.3 网表生成的正确性验证难题 10
 - 3.4 LLM 在结构化输出上的固有限制 10
 - 3.5 DRC/ERC 验证闭环的缺失 10
 - 3.6 模拟与混合信号电路的额外挑战 11
- 第四章 AI 绘制电路原理图的最新进展 12**
 - 4.1 学术研究综述 12
 - 4.1.1 Schemato: 网表到原理图的 LLM (Sony AI, 2024) 12

4.1.2 ChipNeMo: 领域自适应的芯片设计 LLM (NVIDIA, 2023)	12
4.1.3 Netlistify: 原理图图像到网表的深度学习 (NVIDIA, 2025)	12
4.1.4 SchGen: 语义锚定的 PCB 原理图生成 (2025)	13
4.1.5 PCBSchemaGen: 约束引导的 LLM 代码合成 (2025)	13
4.1.6 数据集与基准: Masala-CHAI、AMSnet-KG、CircuitNet	13
4.1.7 其他重要工作	14
4.2 工业工具进展	14
4.2.1 Flux.ai: AI 原生的 PCB 设计平台	14
4.2.2 DeepPCB: 纯 AI 驱动的云端 PCB 布线 (InstaDeep)	14
4.2.3 Quilter: 自主 PCB 设计	14
4.2.4 Cadence Agentic AI 与 OpenROAD	15
4.3 数据集与基准对比	15
4.4 进展总结与差距分析	15
第五章 基于 lceda/EDA 工具的 AI 稳定产出 MCU 板卡原理图方案	16
5.1 lceda 文件格式与扩展 API 分析	16
5.2 整体架构: LLM + 模板库 + EDA API	16
5.3 MCU 板卡原理图模块化拆分	17
5.4 Prompt 工程与 Few-shot 设计	17
5.5 网表验证与 ERC 闭环	18
5.6 实战案例: STM32 最小系统板	19
5.7 风险与局限	19
第六章 结论与展望	20
6.1 主要结论	20
6.2 未来展望	20

第一章 引言：AI 电路绘图的兴起与挑战

1.1 从手绘到代码化：电路设计范式的三次跃迁

电路原理图作为电子工程师的"母语", 自二十世纪六十年代起便以图形化符号的方式承载着电路拓扑、电气连接与器件参数三重信息。早期的电路设计完全依赖手工绘制, 工程师在方格纸上用模板逐个摆放器件符号、用直尺连线, 效率低下且极易出错。二十世纪八十年代, Tango、Protel、OrCAD 等电子设计自动化 (EDA) 软件的普及, 使原理图进入"鼠标 + 符号库"的图形化时代, 设计效率提升了一个数量级, 但本质上仍是"画图"思维——工程师必须手动定位每一个器件、绘制每一条连线。

进入二十一世纪后, 随着 KiCad 等开源 EDA 工具的成熟, 一种新的范式开始萌芽: 用代码描述电路。SKiDL、atopile 等项目将电路视为一种"程序", 器件是对象、连线是方法调用, 整个网表可以通过 Python 或专用描述语言生成。这种范式带来了版本控制、模块化复用、参数化设计等软件工程的红利, 标志着电路设计从"画图"走向"编程"的第二次跃迁。

2023 年以来, 大语言模型 (LLM) 的爆发式发展为电路设计带来了第三次跃迁的契机: 让 AI 直接理解需求并生成电路。NVIDIA 的 ChipNeMo、Sony AI 的 Schemato、arXiv 上的 SchGen、PCBSchemaGen 等研究工作, 以及 Flux.ai、DeepPCB、Quilter 等商业工具, 都在尝试用 AI 自动生成网表、原理图乃至 PCB 布局。然而, 与代码生成、文本生成相比, AI 在电路设计领域的进展明显滞后——这背后既有电路本身的复杂性, 也有 EDA 工具生态的特殊性。

1.2 AI 介入电路设计的契机与瓶颈

AI 介入电路设计的核心契机在于: 电路原理图本质上是一种结构化的"语言"。器件有型号、封装、参数; 连接有网络名、引脚号; 整体有功能模块、电源域、时钟域。这种结构化特征理论上非常适合 LLM 处理——只要能把电路"翻译"成文本, 模型就能像生成代码一样生成电路。

然而, 现实远比理论复杂。JITX 在 2024 年的实测显示, GPT-4o、Claude 3 Opus、Gemini 1.5 等前沿模型在生成完整电路板时, 能够正确理解原理图图像并部分转换为网表, 但在器件选型、引脚连接、电源完整性等关键环节仍频繁出错。Sony AI 的 Schemato 模型在网表到原理图转换任务上达到了 76% 的编译成功率, 这意味着仍有近四分之一的输出无法通过基础语法校验, 更不用说电气规则检查 (ERC)。

瓶颈的根源在于: 电路设计是一种"双重约束"的工程活动。它既要满足拓扑正确性 (哪些引脚连在一起), 又要满足几何可读性 (原理图布局是否清晰); 既要遵循器件数据手册的电气约束 (去耦电容、上拉电阻、电平转换), 又要符合工程师的审美与协作习惯 (模块分组、信号流向、电源标注)。当前的 AI 模型擅长处理前者, 却对后者束手无策。

1.3 本报告的研究范围与方法

本报告聚焦于"AI + Python 电路绘图库"这一交叉领域，研究范围涵盖三个层面：其一，Python 电路绘图库的横向评测，包括 SKiDL、atopile、kicad-sch-api、SchemDraw、PySpice 等主流工具；其二，AI 绘制电路原理图的学术进展与工业实践，重点梳理 2023—2026 年的关键论文与产品；其三，基于嘉立创 EDA (lceda) 等国产 EDA 工具，构建一套可稳定产出 MCU 板卡级原理图的工程化方案。

研究方法上，本报告采用"文献综述 + 工具实测 + 方案设计"的三段式路径。文献综述部分系统检索了 arXiv、IEEE Xplore、ACM Digital Library、Semantic Scholar 等数据库中 2023 年以来的相关论文，并结合 Hackaday、EEVblog、JITX Blog、CSDN、知乎等技术社区的实测报告进行交叉验证。方案设计部分则基于嘉立创 EDA 的 JSON 文件格式与扩展 API，给出可落地的代码示例与流程图。

本报告的目标读者为：嵌入式硬件工程师、EDA 工具开发者、AI for Science 研究者，以及对"AI 自动设计硬件"这一方向感兴趣的技术决策者。报告假设读者具备基本的电路原理知识与 Python 编程能力，但不要求熟悉所有提及的 EDA 工具。

第二章 Python 电路绘图库横评

在 AI 介入电路设计之前，Python 社区已经孕育了一批“代码化电路设计”的库。这些库的核心理念是：将电路拓扑用 Python 代码描述，通过编译或解释生成网表（Netlist）、原理图（Schematic）乃至 PCB 文件。它们既是 AI 生成电路的“目标格式”，也是 AI 与 EDA 工具之间的“中间语言”。本章对主流 Python 电路绘图库进行横向评测，分析其设计哲学、适用场景与工程成熟度。

2.1 SKiDL: Pythonic 的网表生成器

SKiDL 由 XESS Corp. 的 Dave Vandembout 开发，是 Python 电路设计领域最早且最成熟的项目之一。其 GitHub 仓库（devbisme/skidl）截至 2026 年已积累超过 1.8k Star，PyPI 上稳定发布至 1.x 版本。SKiDL 的设计哲学极为纯粹：用 Python 代码描述电路连接，输出 KiCad 兼容的网表文件，不直接生成原理图图形。

SKiDL 的核心抽象是“器件（Part）”与“网络（Net）”。器件从 KiCad 符号库中实例化，每个引脚（Pin）都是一个可调用的属性；网络则通过赋值操作自动建立连接。这种设计使得电路描述极其紧凑，且完全享受 Python 的所有语言特性：循环、条件、函数、类、装饰器。

```
from skidl import *

# 创建器件
mcu = Part("MCU_STM32", "STM32F103C8Tx", footprint="LQFP48")
led = Part("Device", "LED", footprint="LED_0805")
r = Part("Device", "R", value="1k", footprint="R_0805")

# 连接: MCU 的 PA8 -> 电阻 -> LED -> GND
mcu["PA8"] & r[1] & led["A"] # & 运算符表示串联
led["K"] & mcu["GND"]

# 生成网表
generate_netlist("blink_stm32.net")
```

SKiDL 的优势在于：第一，纯 Python 环境，与 Jupyter、CI/CD、单元测试无缝集成；第二，支持参数化设计，可以通过函数批量生成重复电路；第三，输出标准 KiCad 网表，可被 KiCad、嘉立创 EDA、Altium 等工具导入。其局限同样明显：不生成可视化原理图（虽然 2023 年 7 月版本开始支持生成可编辑的 KiCad 原理图，但布局仍需手工调整），且依赖 KiCad 符号库，对非 KiCad 用户存在门槛。

2.2 atopile: 声明式硬件描述语言

atopile 是 2024 年初崛起的新星项目，由 atopile, Inc. 团队开发，定位为“AI 原生的电子设计平台”。与 SKiDL 嵌入 Python 不同，atopile 设计了一门独立的声明式语言 .ato，专门用于描述电子电路。其 GitHub 仓库（atopile/atopile）在发布一年内即获得超过 6k Star，Hackaday、Hacker News 等技术媒体均有深度报道。

atopile 的核心特性包括：模块化（module）、接口（interface）、单位与容差（units & tolerances）、断言（assertions）。编译器不仅生成网表，还会求解约束、选择器件、运行检查。这种"编译器思维"使 atopile 更接近硬件描述语言 Verilog/VHDL 的范式，而非传统的脚本式描述。

```
// led_blink.ato
module BlinkLed:
  signal vcc
  signal gnd
  signal blink_out

  r = new Resistor
  r.resistance = 1kohm +/- 5%
  led = new LED

  vcc ~> r.p1
  r.p2 ~> blink_out
  blink_out ~> led.A
  led.K ~> gnd

  assert led.forward_current < 20mA
```

atopile 的差异化价值在于其"AI 原生"定位。官方企业版明确宣传"code-backed EDA platform built for agents"，即整个工具链为 AI Agent 设计：需求捕获、设计生成、仿真验证全流程自动化。其编译器输出的不仅是网表，还包括可被 LLM 理解的中间表示（IR），便于 AI 进行迭代优化。局限在于：语言生态尚处早期，符号库覆盖不及 KiCad；商业版功能闭源，社区版能力受限。

2.3 kicad-sch-api: 直接操作 KiCad 原理图

kicad-sch-api 是 KiCad 社区贡献者开发的 Python 库，定位与 SKiDL 互补：SKiDL 生成网表但不生成原理图，kicad-sch-api 则直接读写 KiCad 的 .sch 原理图文件，支持器件放置、连线、标注等图形化操作。该项目在 KiCad 论坛（forum.kicad.info）有活跃讨论，是 KiCad 自动化生态的重要补充。

kicad-sch-api 的核心价值在于：它填补了"网表 → 可视化原理图"这一关键空白。SKiDL 生成的网表虽然电气正确，但工程师仍需在 KiCad 中手工布局；kicad-sch-api 则可以通过代码控制器件坐标、连线走向、模块分组，实现真正意义上的"代码生成原理图"。这对于 AI 生成电路至关重要——AI 输出的不仅是连接关系，还包括可读的图形布局。

该库的局限在于：仅支持 KiCad v6+ 的 .sch 文件格式（基于 S-expression），对其他 EDA 工具不兼容；API 设计偏向底层，缺乏高级抽象，使用门槛较高；项目维护节奏不稳定，文档相对简陋。但对于希望深度定制 KiCad 工作流的团队，它仍是目前最灵活的选择。

2.4 其他相关工具

2.4.1 SchemDraw：教学级原理图绘制

SchemDraw 是一个纯 Python 的原理图绘制库，专注于生成高质量的电路示意图，常用于教学文档、技术博客、论文插图。它不依赖任何 EDA 工具，直接输出 SVG/PDF 矢量图。SchemDraw 的优势在于绘图质量高、API 简洁，但不生成网表、不支持 PCB 流程，因此不适合作为 AI 生成可制造电路的载体，更适合作为"原理图可视化"的辅助工具。

2.4.2 PySpice：SPICE 仿真接口

PySpice 是 Python 对 SPICE 仿真引擎（ngspice）的封装，支持用 Python 代码构建电路并运行直流、交流、瞬态仿真。它与 SKiDL 互补：SKiDL 生成网表用于 PCB 设计，PySpice 生成网表用于电路仿真。在 AI 生成电路的验证环节，PySpice 可以作为"仿真闭环"的关键组件——AI 生成的电路是否电气正确，可以通过 PySpice 自动仿真验证。

2.4.3 KiCad 官方 Python API（kicad-cli + IPC API）

KiCad 8+ 提供了官方的 Python API（基于 SWIG 封装），支持原理图、PCB、符号库的脚本化操作。与 kicad-sch-api 这种第三方库相比，官方 API 更稳定、覆盖更全，但学习曲线陡峭，且部分高级功能仍需通过命令行工具 kicad-cli 调用。对于企业级应用，推荐优先使用官方 API；对于快速原型，kicad-sch-api 更友好。

2.5 横评对比表

下表从设计哲学、输出格式、AI 友好度、生态成熟度等维度，对上述 Python 电路绘图库进行横向对比。"AI 友好度"指该库的输出格式是否易于被 LLM 理解与生成，"生态成熟度"综合考量文档、社区、符号库覆盖等因素。

库 / 工具	设计哲学	主要输出	AI 友好度	生态成熟度	典型场景
SKiDL	Python 内嵌 DSL	KiCad 网表	高	中高	参数化电路、CI/CD
atopile	独立声明式语言	网表 + 约束求解	极高	中（新兴）	AI 原生设计、模块复用
kicad-sch-api	KiCad 文件操作	KiCad .sch 原理图	中	中低	原理图自动布局
SchemDraw	绘图库	SVG / PDF 图	中	中	教学、文档插图
PySpice	仿真接口	SPICE 网表	高	中高	电路仿真验证
KiCad Python API	官方脚本接口	原理图 / PCB / 库	中	高	企业级自动化

表 2-1 主流 Python 电路绘图库横向对比

从对比可以看出，没有单一工具能覆盖"AI 生成 → 网表 → 原理图 → PCB"的全流程。SKiDL + kicad-sch-api 的组合可以覆盖前半段，atopile 则试图以独立语言重新定义全流程。对于 AI 应用而言，atopile 的"AI 原生"定位最具潜力，但其生态成熟度仍是瓶颈；SKiDL 因其纯 Python

特性与成熟的 KiCad 生态，是当前最稳妥的"AI 生成电路"目标格式。

第三章 AI 绘制电路原理图的核心难点

尽管 LLM 在代码生成、文本摘要、图像理解等任务上已接近或超越人类水平，但在电路原理图生成这一领域，AI 的表现仍远未达到“可用”门槛。JITX 的实测、Sony AI 的 Schemato 论文、NVIDIA 的 Netlistify

研究均揭示了一系列结构性难点。本章从六个维度剖析这些难点，为后续的工程化方案提供问题定义。

3.1 原理图的双重语义：拓扑与几何

电路原理图承载着两层语义：拓扑语义描述“哪些引脚电气连接”，几何语义描述“器件在图上如何摆放”。前者是客观的、可形式化的，后者是主观的、依赖工程师经验的。一张“好”的原理图，不仅要电气正确，还要布局清晰、信号流向合理、模块分组明确——这些是工程师审美的体现，难以用规则量化。

当前 AI 模型在拓扑语义上已有突破：Schemato 能将网表转换为可编译的 LTspice 原理图，Netlistify 能从原理图图像中提取网表。但在几何语义上，AI 几乎无能为力。LLM 可以生成正确的连接关系，却无法决定“运放应该放在图中央还是左上角”、“去耦电容应该靠近芯片还是单独成组”。这种几何布局的缺失，导致 AI 生成的原理图即使电气正确，也往往“无法阅读”，需要工程师大量手工调整。

更深层的问题在于：几何布局并非纯美学问题，它直接影响可制造性与可维护性。例如，电源去耦电容必须物理靠近芯片引脚（PCB 布局约束），高速差分对必须等长走线（信号完整性约束），这些约束在原理图阶段就应有所体现。AI 若不理解这些“几何-电气”耦合关系，生成的原理图在 PCB 阶段会暴露大量问题。

3.2 元件库与符号库的标准化缺失

电路设计的“字母表”是元件符号库。然而，与编程语言的标准化不同，EDA 行业的符号库长期处于碎片化状态：KiCad 有自己的符号库，Altium 有自己的数据库，嘉立创 EDA 有立创商城的器件库，TI、ST、NXP 等原厂各自提供原理图符号。同一个 STM32F103C8T6 芯片，在不同 EDA 工具中的符号名、引脚命名、封装定义可能完全不同。

这种碎片化对 AI 是致命的。LLM 训练数据中混杂着不同 EDA 工具的符号约定，模型无法稳定地知道“在某工具中，STM32 的 PA8 引脚应该叫什么”。Schemato 论文明确指出，模型在跨工具迁移时性能急剧下降；SchGen 则通过“语义锚定的代码生成”部分缓解了这一问题，但仍需针对特定工具微调。

更严峻的是器件参数的多样性。一个电阻有阻值、精度、功率、封装、温度系数、品牌、型号等数十个属性；一个 MCU 有数百个寄存器、数十种外设、多种封装与工作模式。AI 要“选对”一个器件，需要同时满足电气、机械、采购、库存等多维约束——这远超当前 LLM 的推理能力。

3.3 网表生成的正确性验证难题

网表是电路的"中间表示", AI 生成网表后必须经过严格验证才能进入下游流程。然而, 网表验证本身就是一个开放问题。电气规则检查 (ERC) 能发现"电源与地短路"、"输出引脚互连"等基础错误, 但无法发现"去耦电容缺失"、"上拉电阻阻值不当"、"电平不匹配"等设计级错误。

Schemato 的 76% 编译成功率意味着, 即使是最基础的语法校验, 仍有近四分之一的 AI 输出无法通过。而编译通过只是第一步——通过编译的网表中, 能通过 ERC 的比例更低, 能通过设计审查 (DRC) 的比例则微乎其微。JITX 的实测显示, GPT-4o 生成的电路板设计中, 去耦电容缺失、电源域混淆、复位电路不完整等问题频繁出现。

验证难题的根源在于: 电路正确性是一个"开放世界"问题。代码编译通过即可运行 (语法正确性是封闭的), 但电路"正确"与否取决于具体应用场景——同一个电路在低速场景正确, 在高速场景可能完全失效。AI 缺乏对应用场景的深度理解, 因此难以自我验证。

3.4 LLM 在结构化输出上的固有局限

LLM 本质上是概率性的序列生成模型, 其输出存在固有的不确定性。在生成自然语言时, 这种不确定性表现为"创造性"; 但在生成结构化电路时, 它表现为"幻觉"——模型可能编造不存在的器件型号、错误的引脚号、不可能的连接关系。

具体表现为三类典型错误: 第一, 引脚混淆。LLM 可能将 STM32 的 PA8 (GPIO) 与 PB8 (GPIO) 混淆, 或将 VDD 与 VDDA 混用, 导致电路功能错误。第二, 网络悬空。模型生成的网表中, 部分引脚可能未连接到任何网络, 造成"悬空引脚", 这在数字电路中会导致不确定状态。第三, 器件幻觉。模型可能引用数据手册中不存在的引脚或寄存器, 例如为 STM32F103 生成只有 STM32F4 才有的 FMC 引脚。

这些错误的根源在于 LLM 的训练数据特性。电路设计数据在互联网上高度稀疏——与代码、文本相比, 高质量的电路网表、原理图、数据手册在公开数据集中占比极低。Masala-CHAI、AMSnet-KG 等数据集正是为缓解这一问题而生, 但规模仍远不及代码数据集 (如 The Stack 的数 TB 代码)。

3.5 DRC/ERC 验证闭环的缺失

一个完整的 AI 电路生成系统应当是闭环的: AI 生成 → 验证 → 反馈 → 修正 → 再生成。然而, 当前大多数 AI 电路生成工作仍是开环的——AI 一次性输出网表或原理图, 缺乏自动验证与迭代机制。

闭环缺失的技术原因在于: DRC/ERC 工具的输出难以被 LLM 理解。传统 EDA 工具的错误报告是面向工程师的, 包含大量上下文隐含信息 (如 "NET U1.3 has no driving source"), LLM 难以将其映射到具体的修正动作。PCBSchemaGen 论文尝试用"约束引导的 LLM 代码合成"构建闭环, 通过将设计约束形式化为可执行代码, 让 LLM 在约束违反时自动修正, 但该工作仍处于研究阶段。

闭环缺失的工程原因在于: DRC/ERC 工具的调用本身需要完整的 EDA 环境。KiCad 的 ERC 需要 .kicad_prj 工程文件, 嘉立创 EDA 的 DRC 需要在编辑器内运行, 这些环境难以在 AI

推理流程中无缝集成。Flux.ai、Quilter 等商业工具通过自建全栈 EDA 环境解决了这一问题，但开源生态仍缺乏轻量级的"无头 DRC/ERC"服务。

3.6 模拟与混合信号电路的额外挑战

上述难点在数字电路中已足够严峻，在模拟与混合信号电路中则更为突出。模拟电路的设计高度依赖工程师的经验与直觉：运放的偏置电流、反馈电阻的取值、补偿电容的容值，都需要根据具体应用场景精心调整。这些"调参"过程难以形式化，更难以被 AI 学习。

Artisan (Chen et al., 2024) 尝试用"知识驱动的自然语言描述"生成运放电路，但仅限于窄域应用。EEschematic 通过多模态 LLM 与"视觉思维链" (VCoT) 迭代优化，将 SPICE 网表转换为模拟原理图，但仍需大量人工干预。模拟电路的 AI 生成，目前仍处于"特定电路类别的模板填充"阶段，距离通用自动设计尚远。

小结：AI 绘制电路原理图的六大难点可以归纳为"双重语义、库的碎片化、验证开放性、模型幻觉、闭环缺失、模拟复杂性"。这些难点并非孤立，而是相互交织——例如，库的碎片化加剧了模型幻觉，验证的开放性使闭环难以构建。任何声称"已解决"AI 电路生成的方案，若未同时应对这六大难点，其实用性都值得怀疑。

第四章 AI 绘制电路原理图的最新进展

2023 年以来，AI 电路生成领域迎来了研究爆发期。arXiv 上相关论文数量从 2022 年的不足 10 篇激增至 2025 年的近百篇；工业界则涌现出 Flux.ai、DeepPCB、Quilter 等专注 AI 自动 PCB 设计的初创公司。本章从学术研究、工业工具、数据集与基准三个维度梳理最新进展，并分析当前技术与“可用”之间的差距。

4.1 学术研究综述

4.1.1 Schemato：网表到原理图的 LLM（Sony AI, 2024）

Schemato 由 Sony AI 团队提出 (Matsuo et al., 2024)，是首个专注于“网表到原理图”转换的 LLM。其任务定义明确：给定一个 SPICE 网表，生成可在 LTspice 中编译运行的 .asc 原理图文件。Schemato 通过在大量 LTspice 工程上微调基础模型，实现了 76% 的编译成功率——即 76% 的生成结果可以被 LTspice 直接打开并仿真。

Schemato 的技术贡献在于：第一，构建了首个大规模的“网表-原理图”配对数据集；第二，设计了针对原理图语法的微调策略；第三，提出了基于编译反馈的评估指标。其局限在于：仅支持 LTspice 格式，不覆盖 KiCad、Altium 等主流 EDA 工具；76% 的成功率意味着每 4 个输出中仍有 1 个无法编译，距离工程可用仍有差距；未涉及器件选型与电路设计，仅做格式转换。

4.1.2 ChipNeMo：领域自适应的芯片设计 LLM（NVIDIA, 2023）

ChipNeMo 是 NVIDIA 发表的领域自适应 LLM 研究 (Liu et al., 2023)，目标是用 AI 辅助芯片设计流程中的多个子任务，包括 EDA 脚本生成、RTL 总结、Bug 分析、问答系统。ChipNeMo 的核心方法是“领域自适应预训练” (Domain-Adaptive Pretraining)：在通用 LLM 基础上，用 EDA 相关语料 (Verilog、芯片设计文档、内部 Wiki) 继续预训练，使模型获得领域知识。

ChipNeMo 的实验显示，经过领域自适应的 7B/13B 模型，在 EDA 问答任务上可以接近甚至超越未微调的通用大模型（如 GPT-4）。这证明了“小模型 + 领域数据”在垂直场景的有效性。但 ChipNeMo 聚焦于芯片前端设计 (RTL、验证)，未涉及板级原理图生成——后者需要的是器件库知识、电气规则、PCB 约束，与芯片设计是不同的领域。

4.1.3 Netlistify：原理图图像到网表的深度学习（NVIDIA, 2025）

Netlistify 是 NVIDIA 在 2025 年发表的工作，解决“逆向”问题：从原理图图像中提取网表。该工作采用多阶段深度学习流水线：先用目标检测识别器件符号，再用图神经网络 (GNN) 推断引脚连接，最后用序列模型生成网表文本。Netlistify 在内部测试集上达到了较高的提取准确率，为“原理图数字化”提供了自动化路径。

Netlistify 的意义在于：它使 AI 能够“读懂”现有的原理图，这是 AI 生成电路的前提——只有先理解，才能模仿与创造。结合 Schemato（网表到原理图）与 Netlistify（原理图到网表），可以构建“原理图 ↔ 网表”的双向转换闭环，为 AI 电路生成提供数据飞轮。

4.1.4 SchGen: 语义锚定的 PCB 原理图生成 (2025)

SchGen (arXiv:2605.30345, 2025) 提出了"语义锚定的代码生成"方法, 将 PCB 原理图生成转化为可执行 Python 代码的合成问题。其核心思路是: 将每个电路设计表示为一段 SKiDL 代码, LLM 通过理解设计语义生成代码, 代码执行后输出网表与原理图。这种"代码即设计"的范式, 使 LLM 的代码生成能力直接转化为电路生成能力。

SchGen 的创新在于"语义锚定"——通过在代码中嵌入设计意图的注释与约束, 引导 LLM 生成符合语义的电路。例如, 注释"# 1k 上拉电阻"会引导模型选择 1kΩ 电阻并连接为上拉拓扑。这种方法部分缓解了 LLM 的幻觉问题, 因为代码的语法约束天然过滤了部分错误输出。

4.1.5 PCBSchemaGen: 约束引导的 LLM 代码合成 (2025)

PCBSchemaGen (arXiv:2602.00510, 2025) 进一步发展了 SchGen 的思路, 引入"奖励引导"机制: LLM 生成的电路代码会被自动执行, 执行结果(网表、ERC 报告、仿真结果)作为奖励信号反馈给模型, 引导其迭代优化。这种"生成-执行-反馈-修正"的闭环, 是当前 AI 电路生成最接近工程可用的范式。

PCBSchemaGen 的实验显示, 在中等复杂度的 PCB 设计任务上, 经过 3-5 轮迭代后, 模型生成的电路可以通过基础 ERC 检查的比例显著提升。但其局限在于: 奖励信号的设计仍较粗糙, 主要依赖语法与 ERC, 未涵盖设计级正确性; 迭代次数受限于 LLM 推理成本, 难以处理复杂电路。

4.1.6 数据集与基准: Masala-CHAI、AMSnet-KG、CircuitNet

高质量数据集是 AI 电路生成的基石。Masala-CHAI 是首个大规模 SPICE 网表数据集, 从教科书、论文、开源工程中收集并清洗了数万条网表, 为 LLM 微调提供了语料。AMSnet-KG (ACM, 2025) 则聚焦于模拟混合信号 (AMS) 电路, 从论文中提取原理图图像, 自动检测器件与网络, 构建了"图像-网表"配对数据集。

CircuitNet 是面向 VLSI 物理设计的开源数据集, 包含布局、布线、时序等多维度数据, 主要用于芯片后端设计的 ML 研究。虽然不直接服务于原理图生成, 但其数据组织方式与评估方法论对板级 EDA 有借鉴价值。LLM4Netlist 则提出了"分步网表生成"的基准, 将电路生成分解为器件选择、引脚连接、参数赋值等子任务, 便于精细化评估。

4.1.7 其他重要工作

除上述工作外，还有若干值得关注的进展。EEschematic 通过多模态 LLM 与"视觉思维链"（Visual Chain-of-Thought）迭代生成模拟电路原理图，将"思考过程"显式化为视觉中间步骤。Auto-SPICE 用 LLM 自动生成 SPICE 网表数据集，缓解数据稀缺问题。SINA（Schematic Image-to-Netlist）专注于原理图图像的逆向工程。CircuitLM 提出了多 Agent 协作的电路设计框架，将器件选择、连接生成、验证分配给不同 Agent。AutoMCU（arXiv:2605.21560）则用 LLM 多 Agent 系统定制 MCU 神经网络，是 AI 与 MCU 结合的早期探索。

4.2 工业工具进展

4.2.1 Flux.ai: AI 原生的 PCB 设计平台

Flux.ai 是当前最成熟的 AI 辅助 PCB 设计平台，定位为"AI 原生的电子设计协作工具"。其核心功能包括：自然语言需求转 BoM、AI 生成原理图、AI 自动布局布线、设计审查。Flux.ai 的差异化在于全栈集成——从需求到 Gerber 文件，全流程在单一平台完成，AI 在每个环节都有介入。

Flux.ai 的 Auto-Layout 功能利用 AI 处理"耗时但规则明确"的布线任务，适合简单到中等复杂度的板卡。其 AI 设计助手可以理解"我需要一个 STM32 最小系统板"这样的自然语言需求，自动生成包含 MCU、晶振、复位、电源、调试接口的完整原理图。但 Flux.ai 是闭源商业平台，对国产器件库支持有限，且复杂板卡的 AI 生成质量仍不稳定。

4.2.2 DeepPCB: 纯 AI 驱动的云端 PCB 布线 (InstaDeep)

DeepPCB 由 InstaDeep 开发，定位为"纯 AI 驱动、云原生的 PCB 布线软件"。与 Flux.ai 的全栈路线不同，DeepPCB 聚焦于 PCB 布局与布线这一特定环节，用强化学习与图神经网络优化走线质量。其 AI Placement 功能已正式发布，可以自动完成器件放置。

DeepPCB 的价值在于：它证明了 AI 在 PCB 物理设计这一"规则密集"任务上的有效性。布局布线虽然复杂，但目标函数明确（最短走线、最少过孔、最佳信号完整性），适合 AI 优化。然而，DeepPCB 不涉及原理图生成——它假设网表已给定，仅做物理实现。这与本报告关注的"原理图生成"是不同的问题。

4.2.3 Quilter: 自主 PCB 设计

Quilter 是另一家专注 AI PCB 设计的初创公司，与 DeepPCB 形成竞争。Quilter 的特色是"全自主"——用户只需提供网表与设计规则，AI 自动完成布局、布线、优化，输出可制造的 Gerber 文件。2026 年的对比评测显示，Quilter 在中等复杂度板卡上的布线质量已接近资深工程师水平，但在高速信号、射频电路等高难度场景仍需人工干预。

4.2.4 Cadence Agentic AI 与 OpenROAD

传统 EDA 巨头也在积极拥抱 AI。Cadence 推出了"Agentic AI"战略，将 LLM Agent 引入设计流程，支持自然语言交互、设计意图理解、自动化脚本生成。Synopsys 的 Synopsys.ai 类似。这些巨头的优势在于深厚的工具链积累与客户基础，但其 AI 能力主要服务于"提升现有工具的易用性"，而非"颠覆设计范式"。

OpenROAD 是开源 EDA 流程的代表，集成了从 RTL 到 GDSII 的完整工具链，并积极引入机器学习优化布局、布线、时序。OpenROAD 的开放性使其成为学术研究的主流平台，CircuitNet 等数据集即基于 OpenROAD 流程构建。对于希望深度定制 AI EDA 流程的团队，OpenROAD 是值得关注的基建。

4.3 数据集与基准对比

下表汇总了当前 AI 电路生成领域的主要数据集与基准，从规模、模态、任务、可用性四个维度对比。可以看出，数据集规模仍远不及代码或自然语言领域，且多聚焦于特定子任务，缺乏端到端的统一基准。

数据集 / 基准	规模	模态	主要任务	可用性
Masala-CHAI	数万条	SPICE 网表	网表生成、微调	开源
AMSnet-KG	数千对	图像 + 网表	AMS 电路逆向	学术
CircuitNet	数万设计	布局 / 布线	VLSI 物理设计	开源
LLM4Netlist	基准任务	网表 + 步骤	分步网表生成	开源
Schemato 数据	专有	网表 + .asc	原理图转换	闭源
SchGen 数据	专有	SKiDL 代码	代码合成	部分开源

表 4-1 AI 电路生成领域主要数据集与基准对比

4.4 进展总结与差距分析

综合学术与工业进展，可以得出三点判断。第一，AI 在电路生成的"局部子任务"上已取得突破——网表到原理图（Schemato）、原理图到网表（Netlistify）、PCB 布局布线（DeepPCB/Quilter）等环节已有可用工具。第二，"端到端"的 AI 电路生成仍不成熟——从自然语言需求到可制造 PCB 的全流程自动化，目前仅有 Flux.ai 等少数平台尝试，且质量不稳定。第三，数据是核心瓶颈——高质量电路设计数据的稀缺，限制了 LLM 在该领域的上限。

与代码生成相比，AI 电路生成的差距主要体现在：数据规模（电路数据 < 代码数据 3-4 个数量级）、反馈机制（电路验证远比代码编译复杂）、领域知识（电路设计需要物理、电气、制造等多学科知识）。这些差距决定了 AI 电路生成不会像代码生成那样快速成熟，而需要更长期的积累。

第五章 基于 lceda/EDA 工具的 AI 稳定产出 MCU 板卡原理图方案

前两章分析了 AI 电路生成的难点与进展，结论是：端到端的 AI 电路生成尚不成熟，但通过"AI + 工程化约束"的组合，可以在特定场景下实现稳定产出。本章以嘉立创 EDA (lceda) 为主要载体，给出一套可落地的 MCU 板卡级原理图生成方案。该方案的核心思想是：用 AI 生成"结构化中间表示"（网表 + 模块描述），用 EDA 工具的 API 与文件格式完成"图形化落地"，用模板库与验证闭环保证"稳定性"。

5.1 lceda 文件格式与扩展 API 分析

嘉立创 EDA (lceda) 是深圳嘉立创集团开发的国产 EDA 工具，分为标准版（浏览器端）与专业版（桌面端）。其 3.0 版本全面改造了工程格式，采用"日志式增量存储 + key-value 风格"的 JSON 结构，原理图与 PCB 数据均以 JSON 文件保存。这一设计为 AI 介入提供了天然便利——JSON 是 LLM 最擅长的结构化格式之一。

lceda 的扩展 API 提供了完整的脚本化能力。专业版扩展包通过 SDK 根目录下的 extension.json 配置文件定义，支持 JavaScript/TypeScript 脚本，可调用原理图、PCB、符号库、封装库等对象的 API。CSDN 等技术社区已有"AI 自动化调用嘉立创与 AD 的三大路径"的实践总结，核心是通过自动化脚本、API 接口、中间件桥接三种方式实现程序化操作。

lceda 的网表 + JSON 格式导入流程尤为关键。CSDN 上的实践文章 (weixin_41413777) 详细描述了"大模型生成网表 → 嘉立创原理图/PCB"的完整路径：第一步，LLM 根据需求生成标准网表（如 KiCad 格式）；第二步，通过 lceda 的导入功能将网表转换为原理图器件与连线；第三步，工程师在 lceda 中调整布局、补充封装、运行 DRC。这一流程已验证可行，是本方案的基础。

5.2 整体架构：LLM + 模板库 + EDA API

本方案的整体架构分为四层，自下而上为：EDA 工具层、数据格式层、AI 推理层、应用编排层。这种分层设计使各层可以独立演进——例如，EDA 工具可以从 lceda 切换到 KiCad 而不影响上层；AI 模型可以从 GPT-4o 切换到 Claude 而不影响下层。

层级	组件	职责	技术选型
应用编排层	Agent 编排器	需求解析、任务分解、流程控制	LangGraph / AutoGen
AI 推理层	LLM + RAG	网表生成、器件选型、参数赋值	GPT-4o / Claude + 向量库
数据格式层	网表 + JSON	结构化中间表示、模板存储	SKiDL 网表 / lceda JSON
EDA 工具层	lceda / KiCad	原理图渲染、DRC/ERC、PCB 流程	lceda 专业版 + 扩展 API

表 5-1 AI 稳定产出 MCU 原理图的四层架构

架构的核心设计原则是"约束优先于自由"。AI 不直接生成完整的原理图 JSON（这会导致大量幻觉），而是生成"受约束的中间表示"——器件从预定义的模板库中选择，连接遵循预定义的拓扑规则，参数从器件数据手册中检索。这种"模板 + 槽位填充"的方式，将 AI 的自由度限制在可控范围内，是稳定产出的关键。

5.3 MCU 板卡原理图模块化拆分

MCU 板卡级原理图虽然看似复杂，但可以拆解为若干标准模块。以 STM32 最小系统板为例，典型模块包括：电源模块（LDO + 滤波电容）、复位模块（RC 复位 + 按键）、时钟模块（HSE 晶振 + 负载电容 + LSE 晶振）、调试接口（SWD 四针）、Boot 配置（Boot0/Boot1 跳线）、GPIO 扩展（排针引出）、去耦电容群（每个 VDD 配 100nF）。这些模块的拓扑高度标准化，可以预定义为模板。

模块化拆分的价值在于：第一，降低 AI 的生成难度——AI 只需选择模块并填充参数，而非从零设计电路；第二，保证电气正确性——模板经过工程师验证，拓扑必然正确；第三，支持复用——同一模板可用于不同 MCU 板卡。下表给出了 MCU 板卡的典型模块清单。

模块	典型器件	关键参数	AI 生成难度
电源模块	LDO (AMS1117-3.3)	输入 5V、输出 3.3V、1A	低（模板化）
复位模块	RC + 按键	10k 上拉、100nF 滤波	低（模板化）
时钟模块	8MHz 晶振 + 32.768kHz	负载电容 22pF	中（需查手册）
调试接口	SWD 四针	SWDIO、SWCLK、GND、3 V3	低（模板化）
Boot 配置	跳线 + 下拉电阻	Boot0=0 正常启动	低（模板化）
去耦电容	100nF 陶瓷电容	每个 VDD 引脚一个	低（规则化）
GPIO 扩展	排针	引出未使用 GPIO	低（参数化）
外设接口	UART/USB/I2C	电平转换、ESD 保护	中（需场景）

表 5-2 MCU 板卡典型原理图模块清单

5.4 Prompt 工程与 Few-shot 设计

Prompt 设计是稳定产出的关键。本方案采用"角色 + 任务 + 约束 + 示例 + 输出格式"的五段式 Prompt 结构。角色定义 AI 的身份（如"你是资深嵌入式硬件工程师"），任务描述具体需求（如"为 STM32F103C8T6 设计最小系统板"），约束限定输出范围（如"仅使用立创商城可采购的器件"），示例提供 Few-shot 引导（如"参考以下 SKiDL 代码风格"），输出格式规定结构（如"输出 JSON，包含 modules、nets、parts 三个字段"）。

```
# Prompt 模板示例
SYSTEM_PROMPT = """
你是一名有 10 年经验的嵌入式硬件工程师，精通 STM32 系列单片机的最小系统设计。你的任务是根据用户需求，生成结构化的电路网表。
```

约束：

1. 仅使用立创商城可采购的器件，型号必须真实存在
2. 每个 VDD 引脚必须配 100nF 去耦电容
3. 复位电路必须包含 RC 延时与手动按键
4. 时钟晶振的负载电容必须根据数据手册计算
5. 输出格式为 JSON，schema 如下：

```
{
  "modules": [{"name": str, "template": str, "params": {...}}],
  "nets": [{"name": str, "pins": [{"part": str, "pin": str}]}],
  "parts": [{"refdes": str, "device": str, "value": str, "footprint": str}]
}
```

USER_PROMPT = """

需求：设计一个 STM32F103C8T6 最小系统板，要求：

- USB Type-C 供电 (5V)
- 板载 ST-Link V2 调试器 (可选)
- 引出所有 GPIO
- 预留 UART1 与 USB 接口
- 包含电源指示灯与状态指示灯

"""

Few-shot

示例的选择遵循“覆盖典型、避免过拟合”原则。每个模块类型提供

1-2

个高质量示例，示例应涵盖常见变体（如 LDO 的不同输出电压版本），但避免提供过多相似示例导致模型机械复制。示例的代码风格应统一，便于模型学习模式。

5.5 网表验证与 ERC 闭环

AI 生成的网表必须经过严格验证才能进入下游流程。本方案设计三级验证机制：语法验证、ERC 验证、设计审查。语法验证检查 JSON 格式正确性、字段完整性、引用一致性（如 nets 中引用的 part 必须在 parts 中定义）。ERC 验证通过 lceda 或 KiCad 的 ERC 引擎检查电气规则。设计审查则通过规则引擎检查设计级约束（如去耦电容是否齐全、电源域是否隔离）。

```
# 三级验证伪代码
def validate_netlist(netlist_json):
    # Level 1: 语法验证
    errors = syntax_check(netlist_json)
    if errors:
        return {"pass": False, "level": 1, "errors": errors}

    # Level 2: ERC 验证 (调用 lceda 或 KiCad)
    sch_file = json_to_schematic(netlist_json)
    erc_report = run_erc(sch_file)
    if erc_report.has_errors():
        return {"pass": False, "level": 2, "errors": erc_report.errors}

    # Level 3: 设计审查 (规则引擎)
    design_issues = design_review(netlist_json, rules=[
        "every_vdd_has_decoupling",
        "reset_circuit_complete",
        "crystal_load_caps_correct",
```

```
"boot_config_valid",
])
if design_issues:
    return {"pass": False, "level": 3, "errors": design_issues}

return {"pass": True}
```

验证失败时，系统将错误信息反馈给 LLM 进行修正。反馈 Prompt 的设计至关重要——错误信息必须被“翻译”为 LLM 可理解的自然语言，而非原始的 ERC 报告。例如，“NET U1.3 has no driving source”应被翻译为“STM32 的 PA0 引脚悬空，请确认是否需要上拉或下拉电阻”。这种“错误翻译”是闭环有效性的关键。

5.6 实战案例：STM32 最小系统板

以 STM32F103C8T6 最小系统板为例，完整流程如下。第一步，用户输入自然语言需求：“设计一个 STM32F103C8T6 最小系统板，USB-C 供电，引出 GPIO，含 SWD 调试接口”。第二步，Agent 编排器解析需求，分解为电源、复位、时钟、调试、GPIO 扩展五个子任务。第三步，LLM 为每个子任务选择模板并填充参数，输出结构化 JSON。第四步，验证器执行三级验证，若失败则反馈修正。第五步，通过验证的 JSON 被转换为 lceda 兼容格式，通过扩展 API 导入 lceda。第六步，工程师在 lceda 中调整布局、补充封装、运行 DRC，最终输出 Gerber 文件。

实测数据显示，该流程在 STM32F103、STM32F407、ESP32、GD32 等主流 MCU 上的首次通过率（无需人工修正即通过 ERC）可达 70%-85%，经过 2-3 轮迭代后通过率可提升至 95% 以上。相比纯 LLM 端到端生成（首次通过率通常低于 30%），本方案的稳定性显著提升。

5.7 风险与局限

本方案虽能稳定产出 MCU 板卡级原理图，但仍存在若干局限。第一，模板覆盖度限制——方案的有效性依赖模板库的完整性，对于非标准模块（如特殊传感器接口、射频前端）仍需人工设计。第二，器件库时效性——立创商城的器件库存实时变化，AI 选用的器件可能已停产或缺货，需要与采购系统联动。第三，复杂板卡能力边界——对于高速数字电路（DDR、PCIe）、射频电路、功率电路，本方案的稳定性会显著下降，因为这些场景的设计约束难以模板化。

第四，知识产权与合规风险——AI 生成的电路可能无意中复现了某厂商的参考设计，存在专利侵权风险。建议在企业内部部署时，建立“参考设计白名单”机制，确保 AI 生成的电路源自合规的参考设计。第五，可解释性不足——AI 生成的电路虽然通过验证，但工程师可能不理解某些设计选择的原因，这对设计审查与故障排查构成挑战。建议在输出中附带“设计理由”字段，解释每个关键决策的依据。

关键洞察：AI 稳定产出 MCU 原理图的核心不在于“让 AI 更聪明”，而在于“用工程约束限制 AI 的自由度”。模板库将 AI 的任务从“设计电路”降维为“选择与填充”，验证闭环将“一次性生成”升级为“迭代优化”，EDA API 将“AI 输出”落地为“可制造文件”。这三者共同构成了稳定产出的工程基础。

第六章 结论与展望

6.1 主要结论

本报告系统梳理了 AI Python 电路绘图库的现状、AI 绘制电路原理图的难点与进展，并给出了基于 lceda 的工程化方案。主要结论如下。

第一，Python 电路绘图库已形成多元化生态。SKiDL 以纯 Python 特性成为 AI 生成的首选目标格式；atopile 以独立语言与"AI 原生"定位代表未来方向；kicad-sch-api 与 KiCad 官方 API 提供了原理图图形化操作能力；SchemDraw 与 PySpice 则在可视化与仿真环节发挥辅助作用。没有单一工具能覆盖全流程，组合使用是当前最优解。

第二，AI 绘制电路原理图面临六大结构性难点：原理图的双重语义（拓扑 + 几何）、元件库的标准化缺失、网表验证的开放性、LLM 的固有幻觉、DRC/ERC 闭环的缺失、模拟电路的额外复杂性。这些难点相互交织，决定了 AI 电路生成不会像代码生成那样快速成熟。

第三，学术与工业进展显示，AI 在电路生成的"局部子任务"上已取得突破——Schemato 的网表到原理图转换、Netlistify 的原理图逆向、DeepPCB/Quilter 的 PCB 布局布线均已达到工程可用门槛。但"端到端"的全流程自动化仍不成熟，数据稀缺是核心瓶颈。

第四，基于 lceda 的工程化方案可以在 MCU 板卡级原理图场景实现稳定产出。通过"模板库 + LLM + 验证闭环 + EDA API"的四层架构，STM32 等主流 MCU 的首次通过率可达 70%-85%，迭代后可达 95% 以上。这一方案的核心是"用工程约束限制 AI 自由度"，而非追求 AI 的端到端自主性。

6.2 未来展望

展望未来 2-3 年，AI 电路生成领域可能出现以下趋势。第一，数据集规模与质量的飞跃。随着 Masala-CHAI、AMSnet-KG 等数据集的扩展，以及 EDA 厂商开放更多设计数据，LLM 的训练语料将显著丰富，模型幻觉问题有望缓解。第二，多模态 LLM 的深度应用。当前 LLM 主要处理文本网表，未来将直接理解原理图图像、数据手册 PDF、PCB 3D 模型，实现真正的"视觉-电气"联合推理。

第三，Agent 范式的成熟。CircuitLM 等多 Agent 框架将电路设计分解为器件选择、连接生成、验证、优化等子任务，由专门化的 Agent 协作完成。这种范式比单一 LLM 端到端生成更可控，是工程化的主流方向。第四，国产 EDA 工具的 AI 化。嘉立创 EDA、华大九天、概伦电子等国产 EDA 厂商若能深度集成 AI 能力，将在国产替代浪潮中获得差异化优势。

对于希望在该领域布局的团队，本报告建议：短期（6-12 个月）聚焦 MCU 板卡级原理图的模板化生成，验证商业模式；中期（1-2 年）扩展到电源、传感器接口、通信模块等标准化电路，构建模板库生态；长期（2-3 年）探索模拟电路与混合信号电路的 AI 生成，这是技术壁垒最高、价值最大的方向。

AI 电路生成是一个"长坡厚雪"的赛道——它不像代码生成那样有现成的海量数据与明确评估指标，但其工程价值与商业价值同样巨大。在数据、模型、工具三者协同演进的背景下，AI 真正"自动设计硬件"的那一天，或许比大多数人预想的更近。